



University of Southern Mississippi

MDP Modeling and Policy Optimization (Frozen Lake Variant)

Author: Tomas Nader

ID: w10172066

CSC648 Deep Reinforcement Learning

Dr. Chaoyang (Joe) Zhang

April 21st, 2026

Table of Contents

1.	Introduction to Digital Reinforcement Learning Concepts.....	3
2.	Part 1: MDP Representation	3
2.1	State Space	3
2.2	Action Space.....	4
2.3	Transition Function	4
2.4	Reward function	4
2.5	Example Transitions.....	4
3.	Part 2: Policy Evaluation	6
3.1	Approach.....	6
3.2	Results.....	7
3.3	Interpretation	9
4.	Part 3: Policy Improvement.....	10
4.1	Approach.....	10
4.2	Results.....	10
4.3	Interpretation	11
5.	Part 4: Value iteration.....	12
5.1	Approach.....	12
5.2	Results.....	12
5.3	Comparison with Part 3	13
5.4	Interpretation	13
6.	Conclusion	13

1. Introduction to Digital Reinforcement Learning Concepts

Reinforcement learning (RL) is a framework where an agent learns how to make decisions by interacting with an environment. At each step, the agent observes its current state, selects an action, and receives a reward. The goal is to maximize the total reward over time.

A common way to model such problems is using a **Markov Decision Process (MDP)**. An MDP is defined by:

- A set of **states (S)**
- A set of **actions (A)**
- A **transition function (P)** that defines how actions lead to new states
- A **reward function (R)** that assigns a value to each transition
- A **discount factor (γ)** that controls how future rewards are valued

The key assumption is the **Markov property**, meaning the next state depends only on the current state and action, not past history.

In this assignment, I apply these concepts to a **Frozen Lake environment**, where the agent must reach a goal while avoiding holes in a stochastic (slippery) grid.

2. Part 1: MDP Representation

2.1 State Space

The environment is a 4×4 grid with 16 states labeled from 0 to 15.

- Start state: 0
- Goal state: 15
- Hole states: 5, 7, 12
- State 11 is a safe (non-terminal) state

All other states are non-terminal.

2.2 Action Space

At each state, the agent can take one of four actions: Left, Down, Right, Up

2.3 Transition Function

The environment is **stochastic (slippery)**. When an action is chosen:

- The intended direction occurs with probability **0.4**
- Each perpendicular direction occurs with probability **0.3**

For example, if the agent chooses **Right**, it may:

- Move Right (0.4)
- Move Up (0.3)
- Move Down (0.3)

If a movement goes outside the grid, the agent stays in the same state. Terminal states (holes and goal) transition to themselves with probability 1.

2.4 Reward function

The reward function is defined as:

$$R(s, a, s') = \begin{cases} 1 & \text{if } s' = 15 \\ 0 & \text{otherwise} \end{cases}$$

This means the agent only receives a reward when it reaches the goal state.

2.5 Example Transitions

Below are example transitions for selected states.

State 4

- Action Right:
 - → State 5 (hole) with probability 0.4
 - → State 0 with probability 0.3
 - → State 8 with probability 0.3

```

State 4
Action Left:
  prob=0.4, next=4, reward=0.0, done=False
  prob=0.3, next=0, reward=0.0, done=False
  prob=0.3, next=8, reward=0.0, done=False
Action Down:
  prob=0.4, next=8, reward=0.0, done=False
  prob=0.3, next=4, reward=0.0, done=False
  prob=0.3, next=5, reward=0.0, done=True
Action Right:
  prob=0.4, next=5, reward=0.0, done=True
  prob=0.3, next=0, reward=0.0, done=False
  prob=0.3, next=8, reward=0.0, done=False
Action Up:
  prob=0.4, next=0, reward=0.0, done=False
  prob=0.3, next=4, reward=0.0, done=False
  prob=0.3, next=5, reward=0.0, done=True

```

Figure 1- State 4 transitions

State 9

- Action Up:
 - → State 5 (hole) with probability 0.4
 - → State 8 with probability 0.3
 - → State 10 with probability 0.3

```

State 9
Action Left:
  prob=0.4, next=8, reward=0.0, done=False
  prob=0.3, next=5, reward=0.0, done=True
  prob=0.3, next=13, reward=0.0, done=False
Action Down:
  prob=0.4, next=13, reward=0.0, done=False
  prob=0.3, next=8, reward=0.0, done=False
  prob=0.3, next=10, reward=0.0, done=False
Action Right:
  prob=0.4, next=10, reward=0.0, done=False
  prob=0.3, next=5, reward=0.0, done=True
  prob=0.3, next=13, reward=0.0, done=False
Action Up:
  prob=0.4, next=5, reward=0.0, done=True
  prob=0.3, next=8, reward=0.0, done=False
  prob=0.3, next=10, reward=0.0, done=False

```

Figure 2 - State 9 transitions

State 14

- Action Right:
 - → State 15 (goal, reward = 1) with probability 0.4
 - → State 10 with probability 0.3
 - → State 14 with probability 0.3

```

State 14
Action Left:
  prob=0.4, next=13, reward=0.0, done=False
  prob=0.3, next=10, reward=0.0, done=False
  prob=0.3, next=14, reward=0.0, done=False
Action Down:
  prob=0.4, next=14, reward=0.0, done=False
  prob=0.3, next=13, reward=0.0, done=False
  prob=0.3, next=15, reward=1.0, done=True
Action Right:
  prob=0.4, next=15, reward=1.0, done=True
  prob=0.3, next=10, reward=0.0, done=False
  prob=0.3, next=14, reward=0.0, done=False
Action Up:
  prob=0.4, next=10, reward=0.0, done=False
  prob=0.3, next=13, reward=0.0, done=False
  prob=0.3, next=15, reward=1.0, done=True

```

Figure 3 - State 14 transitions

These examples demonstrate how uncertainty affects movement.

3. Part 2: Policy Evaluation

3.1 Approach

I was given a fixed policy from the assignment and initialized all state values to zero:

$$V_0(s) = 0$$

I used a discount factor of:

$$\gamma = 0.9$$

I performed **10 iterations** of policy evaluation to observe how values evolve.

3.2 Results

At early iterations, only states close to the goal receive non-zero values. For example:

- After iteration 1, only states that can directly reach the goal have positive values
- As iterations continue, values propagate backward across the grid

By iteration 10, the value function becomes more stable.

Value Function after Iteration 0			
0.000	0.000	0.000	0.000
0.000	0.000	0.000	0.000
0.000	0.000	0.000	0.000
0.000	0.000	0.000	0.000
Value Function after Iteration 1			
0.000	0.000	0.000	0.000
0.000	0.000	0.000	0.000
0.000	0.000	0.000	0.400
0.000	0.000	0.300	0.000
Value Function after Iteration 2			
0.000	0.000	0.000	0.000
0.000	0.000	0.000	0.000
0.000	0.000	0.108	0.508
0.000	0.108	0.408	0.000
Value Function after Iteration 3			
0.000	0.000	0.000	0.000
0.000	0.000	0.029	0.000
0.000	0.068	0.137	0.566
0.000	0.176	0.476	0.000
Value Function after Iteration 4			
0.000	0.000	0.010	0.000
0.000	0.000	0.037	0.000
0.018	0.100	0.182	0.590
0.000	0.237	0.519	0.000
Value Function after Iteration 5			
0.000	0.000	0.013	0.003
0.005	0.000	0.052	0.000
0.032	0.139	0.200	0.608
0.000	0.278	0.551	0.000
Value Function after Iteration 6			
0.001	0.000	0.019	0.005
0.010	0.000	0.058	0.000
0.048	0.163	0.221	0.618
0.000	0.311	0.573	0.000
Value Function after Iteration 7			
0.003	0.000	0.022	0.009
0.017	0.000	0.065	0.000
0.061	0.185	0.232	0.626
0.000	0.334	0.590	0.000
Value Function after Iteration 8			
0.006	0.001	0.026	0.011
0.023	0.000	0.068	0.000
0.072	0.199	0.242	0.632
0.000	0.353	0.603	0.000
Value Function after Iteration 9			
0.008	0.002	0.028	0.014
0.029	0.000	0.072	0.000
0.082	0.212	0.249	0.636
0.000	0.366	0.612	0.000
Value Function after Iteration 10			
0.011	0.004	0.031	0.017
0.035	0.000	0.075	0.000
0.090	0.221	0.255	0.639
0.000	0.376	0.619	0.000

Figure 4 - Policy evaluation

The policy evaluation converges after 22 iterations showing the following:

- Higher values near the goal
- Lower values near holes
- Gradual increase toward the goal

Converged Value Function under Original Policy			
0.030	0.014	0.042	0.030
0.064	0.000	0.085	0.000
0.125	0.255	0.275	0.649
0.000	0.410	0.641	0.000
Policy evaluation converged in 22 iterations.			

Figure 5 - Converged value table

3.3 Interpretation

The value function represents the expected return when following the given policy.

Key observations:

- Values **propagate backward** from the goal because rewards are delayed
- States closer to the goal have higher values
- States near holes have lower values due to risk
- Stochastic transitions reduce certainty, lowering overall values

This demonstrates how uncertainty affects expected outcomes.

4. Part 3: Policy Improvement

4.1 Approach

Using the converged value function, I computed the **action-value function**:

$$Q(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$$

For each state, I selected the action with the highest Q-value to create an improved policy.

4.2 Results

The improved policy differed from the original in the following states: 0, 1, 2, 6, 10, 14

```
[0, 1, 2, 6, 10, 14]
State 0: Right -> Left
State 1: Left -> Up
State 2: Down -> Right
State 6: Right -> Down
State 10: Up -> Down
State 14: Down -> Right
```

Figure 6 - Difference in states

This shows that the original policy was not optimal.

```
Improved Policy
L   U   R   U
L   H   D   H
U   D   D   D
H   R   R   G
```

Figure 7 - Original Policy

Q-values by state				
State 0:	L=0.0364	D=0.0352	R=0.0307	U=0.0229
State 1:	L=0.0147	D=0.0196	R=0.0191	U=0.0248
State 2:	L=0.0397	D=0.0427	R=0.0452	U=0.0272
State 3:	L=0.0233	D=0.0195	R=0.0188	U=0.0303
State 4:	L=0.0651	D=0.0624	R=0.0419	U=0.0282
State 6:	L=0.0856	D=0.0988	R=0.0856	U=0.0153
State 8:	L=0.0624	D=0.1026	R=0.1091	U=0.1257
State 9:	L=0.1557	D=0.2554	R=0.2095	U=0.1079
State 10:	L=0.2879	D=0.4750	R=0.4299	U=0.2749
State 11:	L=0.3988	D=0.6494	R=0.5337	U=0.2494
State 13:	L=0.1795	D=0.3207	R=0.4103	U=0.2649
State 14:	L=0.3948	D=0.6415	R=0.6473	U=0.5095

Figure 8 - Q-values by state

4.3 Interpretation

The improved policy changes decisions mainly in:

- Early states (far from the goal)
- States near holes

Key insight:

- Even if an action moves toward the goal, it may not be optimal if there is a high probability of slipping into a hole.

Because of stochastic transitions:

- The agent prefers **safer paths**
- It avoids risky moves even if they are shorter

This highlights the importance of **expected value over immediate direction**.

5. Part 4: Value iteration

5.1 Approach

I implemented value iteration using the Bellman optimality equation:

$$V(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$$

I initialized all values to zero and updated them iteratively until:

$$\max |V_{new} - V_{old}| < 0.001$$

5.2 Results

Value iteration converged in 27 iterations and the final value function had higher values than in Part 2. Also, the optimal policy was computed from the final values.

```
Optimal Value Function from Value Iteration
0.098  0.094  0.127  0.091
0.131  0.000  0.214  0.000
0.215  0.408  0.594  0.767
0.000  0.529  0.767  0.000

Optimal Policy from Value Iteration
D  U  D  U
L  H  D  H
U  D  D  D
H  R  R  G

Value iteration converged in 27 iterations.
```

Figure 9 - Optimal Value Function and Optimal Policy

5.3 Comparison with Part 3

The optimal policy differed from the improved policy in:

- State 0
- State 2

This shows that policy improvement gives a **better policy**, but not necessarily optimal and value iteration finds the **true optimal policy**.

```
Comparison: Part 3 improved policy vs Part 4 optimal policy
States that differ:
[0, 2]
State 0: Left -> Down
State 2: Right -> Down
```

Figure 10 - Comparison

5.4 Interpretation

The optimal policy:

- Moves efficiently toward the goal
- Avoids risky transitions near holes
- Balances reward and uncertainty

Compared to Part 3, it makes more globally optimal decisions rather than local improvements.

6. Conclusion

In this assignment, I modeled the Frozen Lake environment as an MDP and applied key reinforcement learning techniques.

What was done:

- Defined the MDP components (states, actions, transitions, rewards)
- Evaluated a fixed policy and observed value propagation
- Improved the policy using action-values

- Computed the optimal policy using value iteration

Key takeaways:

- Rewards propagate backward through the value function
- Stochastic environments require reasoning about probabilities
- Optimal decisions are based on expected outcomes, not direct paths
- Value iteration provides a globally optimal solution

This assignment demonstrates how uncertainty significantly affects both value estimation and policy decisions.